DATA STRUCTURES HOMEWORK

1. (a) Define an integer array. Declare an integer pointer and initialize it to point to the first element of the array. Use pointer arithmetic to traverse the array and print the elements.

```
#include <stdio.h>
int main()
{
   int arr[5] = {10, 20, 30, 40, 50};
   int *p = arr;
   int i;
   printf("Array elements:\n");
   for(i = 0; i < 5; i++)
   {
      printf("%d ", *(p + i));
   }
   return 0;
}
```

OUTPUT:

```
Array elements:
10 20 30 40 50
```

(b) Modify the elements of the array using pointer arithmetic.

```
#include <stdio.h>
int main()
{
   int arr[5] = {10, 20, 30, 40, 50};
   int *p = arr;
   int i;
```

```
    for(i = 0; i < 5; i++)
    {
        *(p + i) = *(p + i) + 5;
    }
    printf("Modified array:\n");
    for(i = 0; i < 5; i++)
    {
        printf("%d ", *(p + i));
    }
    return 0;
}
```

OUTPUT:

Modified array:

15 25 35 45 55

2.Calculate string length using pointers.

```
#include <stdio.h>
int main()
{
    char str[100];
    char *p;
    int length = 0;
    printf("Enter a string: ");
    scanf("%[^\n]", str);
    p = str;
    while(*p != '\0')
    {
        length++;
```

```
        p++;
    }
    printf("Length = %d", length);
    return 0;
}
```

OUTPUT:

Enter a string: jothi mani

Length = 10

---

3. Write a function that calculates both the sum and product of two numbers. Return these two results using pointers passed as arguments to the function.

```
#include <stdio.h>
void calculate(int a, int b, int *sum, int *product)
{
    *sum = a + b;
    *product = a * b;
}
int main()
{
    int a, b;
    int sum, product;
    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);
    calculate(a, b, &sum, &product);
    printf("Sum = %d\n", sum);
    printf("Product = %d", product);
    return 0;
}
```

OUTPUT:

Enter two numbers: 23

34

Sum = 57

Product = 782

---

4. Create an array of structures. Use dynamic memory allocation to allocate memory space and use a structure pointer with pointer arithmetic to iterate and access / modify members.

```
#include <stdio.h>
#include <stdlib.h>
struct Student
{
   int id;
   char name[30];
   float grade;
};
int main()
{
   struct Student *s;
   int n, i;
   printf("Enter number of students: ");
   scanf("%d", &n);
   s = (struct Student *)malloc(n * sizeof(struct Student));

   if(s == NULL)
   {
      printf("Memory allocation failed");
      return 1;
```

```
    }

    for(i = 0; i < n; i++)
    {
        printf("\nEnter details of student %d:\n", i + 1);
        printf("ID: ");
        scanf("%d", &(s + i)->id);
        printf("Name: ");
        scanf("%s", (s + i)->name);
        printf("Grade: ");
        scanf("%f", &(s + i)->grade);
    }

    printf("\nStudent Details:\n");
    for(i = 0; i < n; i++)
    {
        printf("\nID: %d", (s + i)->id);
        printf("\nName: %s", (s + i)->name);
        printf("\nGrade: %.2f\n", (s + i)->grade);
    }
    free(s);
    return 0;
}
```

OUTPUT:

```
Enter number of students: 2

Enter details of student 1:
ID: 101
Name: joe
```

Grade: 80

Enter details of student 2:

ID: 102

Name: mani

Grade: 90

Student Details:

ID: 101

Name: joe

Grade: 80.00

ID: 102

Name: mani

Grade: 90.00

---

5. Define two functions with the same signature. Declare a function pointer and use the pointer to call both functions

```
#include <stdio.h>
void add(int a, int b)
{
   printf("Addition = %d\n", a + b);
}
void multiply(int a, int b)
{
   printf("Multiplication = %d\n", a * b);
}
int main()
```

```
{
    void (*fp)(int, int);
    fp = add;
    fp(10, 5);
    fp = multiply;
    fp(10, 5);
    return 0;
}
```

OUTPUT:

```
Addition = 15
Multiplication = 50
```

---

1. 6. Declare an integer, a pointer to it, and a double pointer. Print the values and addresses at each level.

```
#include <stdio.h>
int main()
{
    int x = 10;
    int *p = &x;
    int **q = &p;

    printf("Value of x = %d\n", x);
    printf("Address of x = %p\n", (void*)&x);
    printf("\nValue of p = %p\n", (void*)p);
    printf("Address of p = %p\n", (void*)&p);
    printf("Value pointed by p = %d\n", *p);
    printf("\nValue of q = %p\n", (void*)q);
    printf("Address of q = %p\n", (void*)&q);
    printf("Value pointed by q = %p\n", (void*)*q);
```

```
    printf("Value of x using **q = %d\n", **q);
    return 0;
}
```

OUTPUT:

Value of x = 10

Address of x = 000000000062FE4C

Value of p = 000000000062FE4C

Address of p = 000000000062FE40

Value pointed by p = 10

Value of q = 000000000062FE40

Address of q = 000000000062FE38

Value pointed by q = 000000000062FE4C

Value of x using **q = 10

---

7. Sort an array of floating point numbers using Selection sort.

```
#include <stdio.h>
int main()
{
    float arr[5] = {4.5, 1.2, 8.7, 2.3, 6.1};
    float temp;
    int i, j, min;

    for(i = 0; i < 4; i++)
    {
        min = i;
        for(j = i + 1; j < 5; j++)
        {
```

```
            if(arr[j] < arr[min])
            {
                min = j;
            }
        }
        temp = arr[i];
        arr[i] = arr[min];
        arr[min] = temp;
    }
    printf("Sorted array:\n");
    for(i = 0; i < 5; i++)
    {
        printf("%.1f ", arr[i]);
    }
    return 0;
}
```

OUTPUT:

Sorted array:

1.2 2.3 4.5 6.1 8.7

---

1. 8. Define a structure Student with id, name, and grade. Create an array of Student structures and sort them based on different fields.

```
#include <stdio.h>
#include <string.h>
struct Student
{
    int id;
    char name[30];
    float grade;
```

```
};
int main()
{
    struct Student s[5], temp;
    int i, j, choice;

    for(i = 0; i < 5; i++)
    {
        printf("\nEnter details of student %d:\n", i + 1);
        printf("ID: ");
        scanf("%d", &s[i].id);
        printf("Name: ");
        scanf("%s", s[i].name);
        printf("Grade: ");
        scanf("%f", &s[i].grade);
    }

    printf("\nSort by:\n");
    printf("1. ID\n");
    printf("2. Name\n");
    printf("3. Grade\n");
    printf("Enter choice: ");
    scanf("%d", &choice);
    for(i = 0; i < 4; i++)
    {
        for(j = i + 1; j < 5; j++)
        {
            int condition = 0;
            if(choice == 1 && s[i].id > s[j].id)
```

```
            condition = 1;
        else if(choice == 2 && strcmp(s[i].name, s[j].name) > 0)
            condition = 1;
        else if(choice == 3 && s[i].grade > s[j].grade)
            condition = 1;
        if(condition)
        {
            temp = s[i];
            s[i] = s[j];
            s[j] = temp;
        }
      }
   }
   printf("\nSorted Students:\n");
   for(i = 0; i < 5; i++)
   {
      printf("%d %s %.2f\n",
           s[i].id, s[i].name, s[i].grade);
   }
   return 0;
}
```

OUTPUT:

Enter details of student 1:

ID: 101

Name: joe

Grade: 80

Enter details of student 2:

ID: 102

Name: mani

Grade: 90

Enter details of student 3:

ID: 103

Name: akil

Grade: 85

Enter details of student 4:

ID: 104

Name: sri

Grade: 95

Enter details of student 5:

ID: 105

Name: bala

Grade: 94

Sort by:

1. ID

2. Name

3. Grade

Enter choice: 1

Sorted Students:

101 joe 80.00

102 mani 90.00

103 akil 85.00

104 sri 95.00

105 bala 94.00

9. Sort a set of negative numbers using radix sort.

```
#include <stdio.h>
int getMax(int arr[], int n)
{
   int max = arr[0];
   int i;
   for(i = 1; i < n; i++)
   {
      if(arr[i] > max)
         max = arr[i];
   }
   return max;
}
void radixPositive(int arr[], int n)
{
   int max = getMax(arr, n);
   int exp = 1;
   int output[100];
   int count[10];
   int i;
   while(max / exp > 0)
   {
      for(i = 0; i < 10; i++)
         count[i] = 0;
      for(i = 0; i < n; i++)
         count[(arr[i] / exp) % 10]++;
      for(i = 1; i < 10; i++)
```

```
        count[i] += count[i - 1];
    for(i = n - 1; i >= 0; i--)
    {
        output[count[(arr[i] / exp) % 10] - 1] = arr[i];
        count[(arr[i] / exp) % 10]--;
    }
    for(i = 0; i < n; i++)
        arr[i] = output[i];
    exp *= 10;
  }
}
int main()
{
  int arr[8] = {-45, 23, -12, 5, -8, 34, -30, 10};
  int pos[8], neg[8];
  int np = 0, nn = 0;
  int i, k = 0;
  for(i = 0; i < 8; i++)
  {
    if(arr[i] < 0)
      neg[nn++] = -arr[i];
    else
      pos[np++] = arr[i];
  }
  if(nn > 0)
    radixPositive(neg, nn);
  if(np > 0)
    radixPositive(pos, np);
  for(i = nn - 1; i >= 0; i--)
```

```
        arr[k++] = -neg[i];
    for(i = 0; i < np; i++)
        arr[k++] = pos[i];
    printf("Sorted array:\n");
    for(i = 0; i < 8; i++)
        printf("%d ", arr[i]);
    return 0;
}
```

OUTPUT:

Sorted array:

-45 -30 -12 -8 5 10 23 34

---

10. Write a C program to copy one array to another using pointers.

```
#include <stdio.h>
int main()
{
    int a[5] = {10, 20, 30, 40, 50};
    int b[5];
    int *p = a;
    int *q = b;
    int i;
    for(i = 0; i < 5; i++)
    {
        *(q + i) = *(p + i);
    }
    printf("Copied array:\n");

    for(i = 0; i < 5; i++)
    {
```

```
        printf("%d ", *(q + i));
    }
    return 0;
}
```

OUTPUT:

Copied array:

10 20 30 40 50